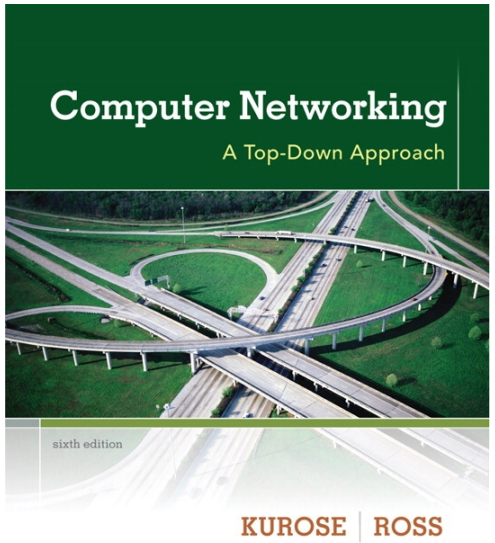
Goal: The following code implements a client-server applications using TCP. Analyse the source code and run the application. Insert comments in code.



Computer Networking: A Top Down Approach

6th edition
Jim Kurose, Keith Ross
Addison-Wesley
March 2012

A note on the use of these ppt slides:

We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

©All material copyright 1996-2012
J.F Kurose and K.W. Ross, All Rights Reserved

Socket programming *with TCP*

TCP: is connection-oriented

- Need for handshake and connection establishment
- One end of the TCP connection is attached to the client socket (IP address+port) and the other end is attached to a server socket
- One side just drops data into the connection via its socket

TCP: data is reliably transmitted

application viewpoint

TCP provides reliable transfer of data stream between client and server

Example: TCP client

from socket import *
serverName = 'servername'
serverPort = 12000
Similar to UDP (see UDP programming lab)
clientSocket = socket(AF_INET, SOCK_STREAM)
Creates the client's socket, called clientSocket. The first parameter indicates that the underlying network is using IPv4. The second parameter indicates that the socket is of type SOCK_STREAM, which means it is a TCP socket (rather than a UDP socket). Port number is created by the operating system.
clientSocket.connect((serverName,serverPort))
Initiates the TCP connection between the client and server. Parameter of the connect() method is the address of the server side of the connection. After this line, three-way handshake is done and a TCP connection is established between the client and server.
sentence = raw_input('Input lowercase sentence:')
Similar to UDP (see UDP programming lab)
clientSocket.send(sentence)
Sends string through the client's socket and into the TCP connection. The program does not explicitly create a packet and attach the destination address to the packet, as was the case with UDP sockets. The client program simply drops the bytes into the TCP connection.
modifiedSentence = clientSocket.recv(1024)
Characters arriving from server are placed into the string modifiedSentence.
print 'From Server:', modifiedSentence
clientSocket.close()

Example: TCP server

```
from socket import *
serverPort = 12000
```
Similar to UDP (see UDP programming lab)
```
serverSocket = socket(AF_INET,SOCK_STREAM)
```
As with TCPClient, the server creates a TCP socket
```
serverSocket.bind((‘’,serverPort))
```
Similarly to UDPServer, we associate the server port number, serverPort, with the created socket. But with TCP, serverSocket will be our welcoming socket. The '' allows the socket to be associated with a specific client IP.
```
serverSocket.listen(1)
```
The server listens for TCP connection requests from the client. The parameter specifies the maximum number of queued connections (at least 1).
```
print ‘The server is ready to receive’
while 1:
    connectionSocket, addr = serverSocket.accept()
```
Program invokes the accept() method for serverSocket, which creates a new socket in the server, called connectionSocket, dedicated to this particular client.
```
    sentence = connectionSocket.recv(1024)
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence)
```
Similar to UDP (see UDP programming lab)
```
    connectionSocket.close()
```